

LIST OF EXPERIMENTS — Sets 11 to 15

R Code, Output Visualization & Python Code

Set 11: Product Category Analysis

Dataset: Product Categories

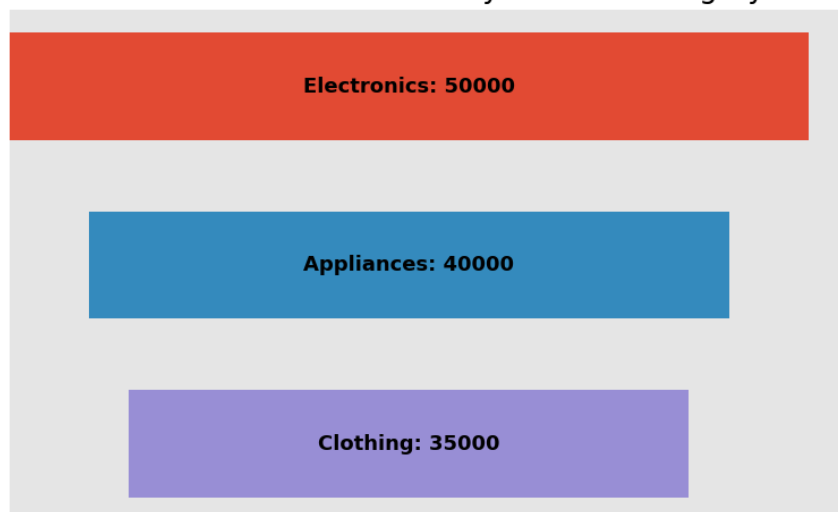
Category	Sales (in \$)
Electronics	50000
Clothing	35000
Appliances	40000

1. Generate a funnel chart to analyze the sales conversion process for each product category. Label the stages and title the chart.

R-code:

```
category <- data.frame(  
  Category = c("Electronics", "Clothing", "Appliances"),  
  Sales = c(50000, 35000, 40000)  
)  
  
library(plotly)  
plot_ly(  
  type = "funnel",  
  y = category$Category,  
  x = category$Sales  
) %>% layout(title = "Sales Conversion Funnel by Product Category")
```

Sales Conversion Funnel by Product Category



Python code:

```
import pandas as pd  
import matplotlib.pyplot as plt  
  
category = pd.DataFrame({  
  "Category": ["Electronics", "Clothing", "Appliances"],  
  "Sales": [50000, 35000, 40000]  
})
```

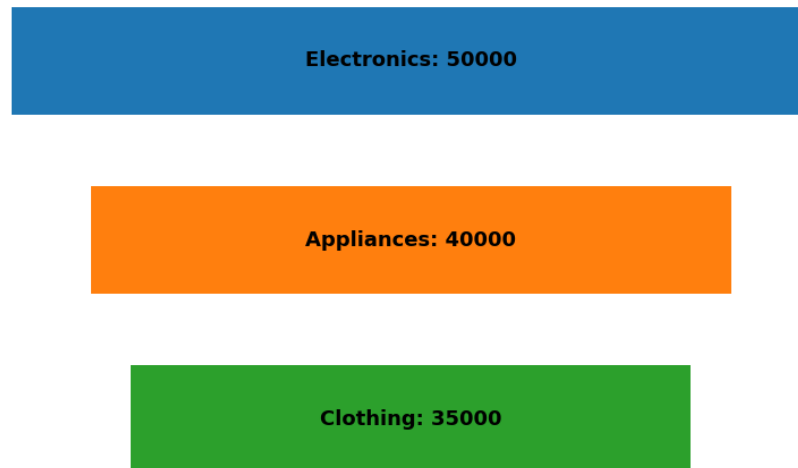
```

}))

df = category.sort_values("Sales", ascending=False)
fig, ax = plt.subplots(figsize=(6,4))
max_val = df["Sales"].max()
for i, row in df.reset_index(drop=True).iterrows():
    width = row["Sales"] / max_val
    ax.barh(i, width, left=(1-width)/2, height=0.6)
    ax.text(0.5, i, f'{row["Category"]}: {row["Sales"]}', ha="center", va="center")
ax.invert_yaxis()
ax.set_xticks([]); ax.set_yticks([])
plt.title("Sales Conversion Funnel by Product Category")
plt.tight_layout()
plt.show()

```

Sales Conversion Funnel by Product Category



2. Build a table to display the sales data for each product category. Label the table elements.

R-code:

```

category <- data.frame(
  Category = c("Electronics","Clothing","Appliances"),
  Sales = c(50000, 35000, 40000)
)
print(category)

```

Output:

```

  Category Sales
1 Electronics 50000
2   Clothing 35000
3 Appliances 40000

```

Python code:

```

import pandas as pd

category = pd.DataFrame({
    "Category": ["Electronics", "Clothing", "Appliances"],
    "Sales": [50000, 35000, 40000]
})
print(category)

```

Output:

```

  Category  Sales
0 Electronics 50000
1   Clothing 35000
2 Appliances 40000

```

3. Develop a Tableau dashboard combining the pie chart, funnel chart, and the table for interactive exploration of product category data.

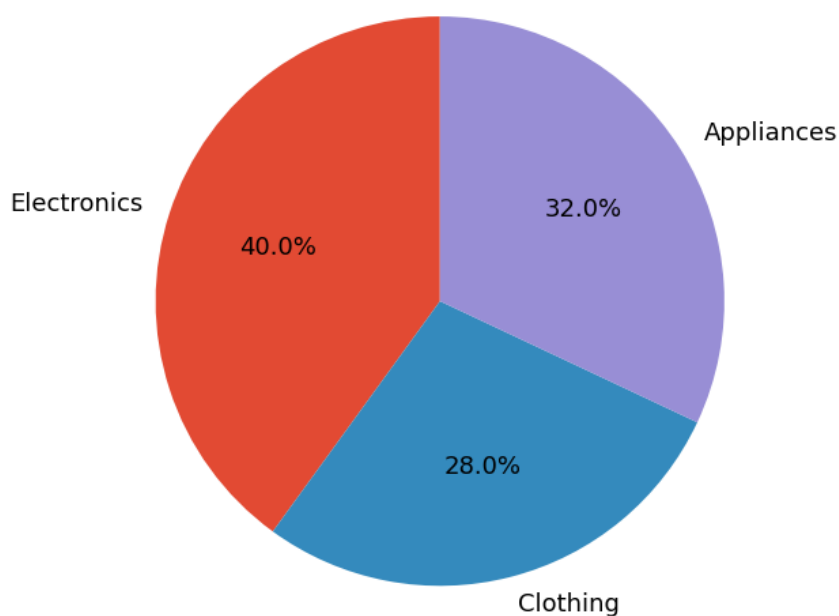
Note: This task is built in Tableau Desktop, not in R or Python. Steps: (1) connect Tableau to the Product Categories data, (2) create a pie-chart sheet of Sales by Category, (3) create a funnel-style sheet (e.g. a bar chart sorted descending, formatted to taper), (4) add a text table sheet of the raw data, and (5) combine all three sheets on one Dashboard with a Category filter.

4. Create a pie chart to represent the distribution of sales across product categories. Include labels.

R-code

```
library(ggplot2)
ggplot(category, aes(x = "", y = Sales, fill = Category)) +
  geom_bar(stat = "identity", width = 1) +
  coord_polar("y") +
  labs(title = "Sales Distribution Across Product Categories") +
  geom_text(aes(label = scales::dollar(Sales)), position = position_stack(vjust = 0.5)) +
  theme_void()
```

Sales Distribution Across Product Categories

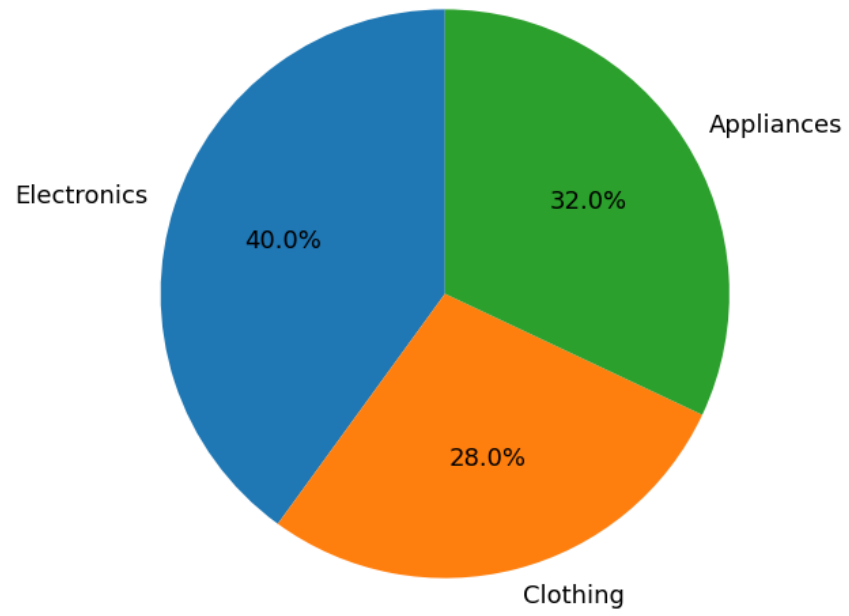


Python code

```
import matplotlib.pyplot as plt

plt.figure(figsize=(5,5))
plt.pie(category["Sales"], labels=category["Category"], autopct="%1.1f%%", startangle=90)
plt.title("Sales Distribution Across Product Categories")
plt.show()
```

Sales Distribution Across Product Categories



Set 12: Website Traffic Analysis

Dataset: Website Traffic

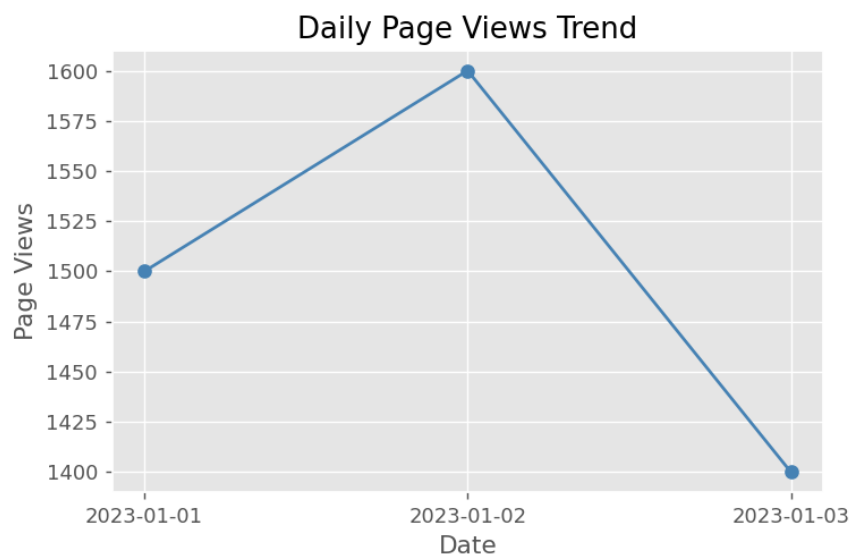
Date	Page Views	Click-through Rate
2023-01-01	1500	2.3%
2023-01-02	1600	2.7%
2023-01-03	1400	2.0%

1. Create a line chart to visualize the trend in daily page views over time. Label the axes and title the chart.

R-code

```
library(ggplot2)
web <- data.frame(
  Date = as.Date(c("2023-01-01", "2023-01-02", "2023-01-03")),
  PageViews = c(1500, 1600, 1400),
  CTR = c(2.3, 2.7, 2.0)
)

ggplot(web, aes(x = Date, y = PageViews)) +
  geom_line(color = "steelblue") +
  geom_point() +
  labs(title = "Daily Page Views Trend", x = "Date", y = "Page Views") +
  theme_minimal()
```



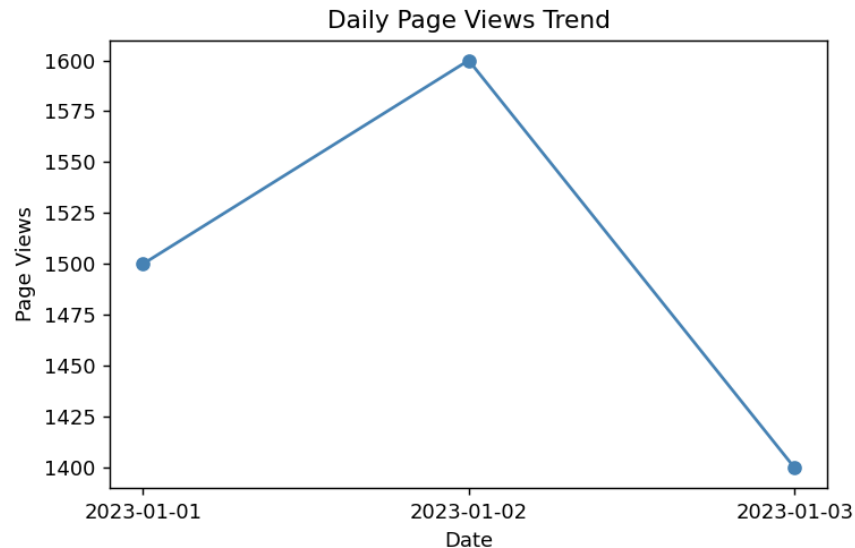
Python code

```
import pandas as pd
import matplotlib.pyplot as plt

web = pd.DataFrame({
    "Date": pd.to_datetime(["2023-01-01", "2023-01-02", "2023-01-03"]),
    "PageViews": [1500, 1600, 1400],
    "CTR": [2.3, 2.7, 2.0]
})

plt.figure(figsize=(6,4))
plt.plot(web["Date"].dt.strftime("%Y-%m-%d"), web["PageViews"], marker="o")
```

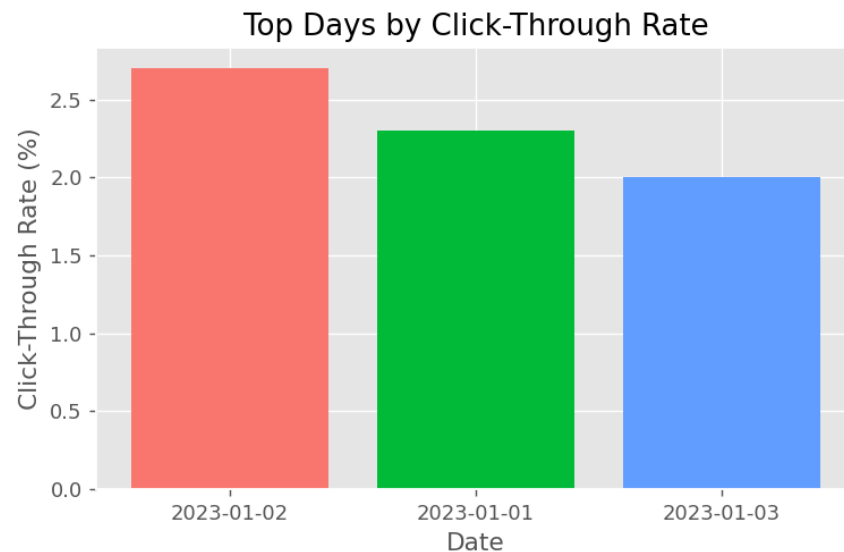
```
plt.title("Daily Page Views Trend")
plt.xlabel("Date")
plt.ylabel("Page Views")
plt.show()
```



2. Generate a bar chart showing the top N days with the highest click-through rates. Label the chart elements.

R-code

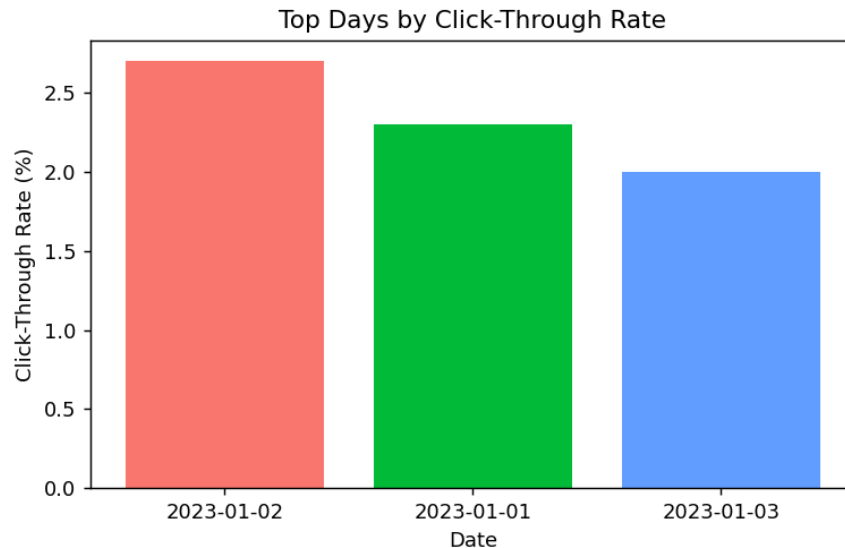
```
top_days <- web[order(-web$CTR), ]
ggplot(top_days, aes(x = reorder(as.character(Date), -CTR), y = CTR, fill = as.character(Date))) +
  geom_bar(stat = "identity") +
  labs(title = "Top Days by Click-Through Rate", x = "Date", y = "CTR (%)") +
  theme_minimal() + theme(legend.position = "none")
```



Python code

```
top_days = web.sort_values("CTR", ascending=False)
plt.figure(figsize=(6,4))
plt.bar(top_days["Date"].dt.strftime("%Y-%m-%d"), top_days["CTR"])
plt.title("Top Days by Click-Through Rate")
plt.xlabel("Date")
plt.ylabel("Click-Through Rate (%)")
```

```
plt.show()
```



3. Build a table to show the website traffic data for each day. Label the table elements.

R-code:

```
print(web)
```

Output:

	Date	PageViews	CTR
1	2023-01-01	1500	2.3
2	2023-01-02	1600	2.7
3	2023-01-03	1400	2.0

Python code:

```
print(web)
```

Output:

	Date	PageViews	CTR
0	2023-01-01	1500	2.3
1	2023-01-02	1600	2.7
2	2023-01-03	1400	2.0

4. Develop a Tableau dashboard combining the line chart, bar chart, and the table for interactive exploration of website traffic data.

Note: This task is built in Tableau Desktop, not in R or Python. Steps: (1) connect Tableau to the Website Traffic data, (2) build a line-chart sheet of Date vs Page Views, (3) build a bar-chart sheet of Date vs CTR, (4) add a text table sheet of the raw data, and (5) combine all three on one Dashboard.

Set 13: Geographic Data Analysis

Dataset: Geographic Data

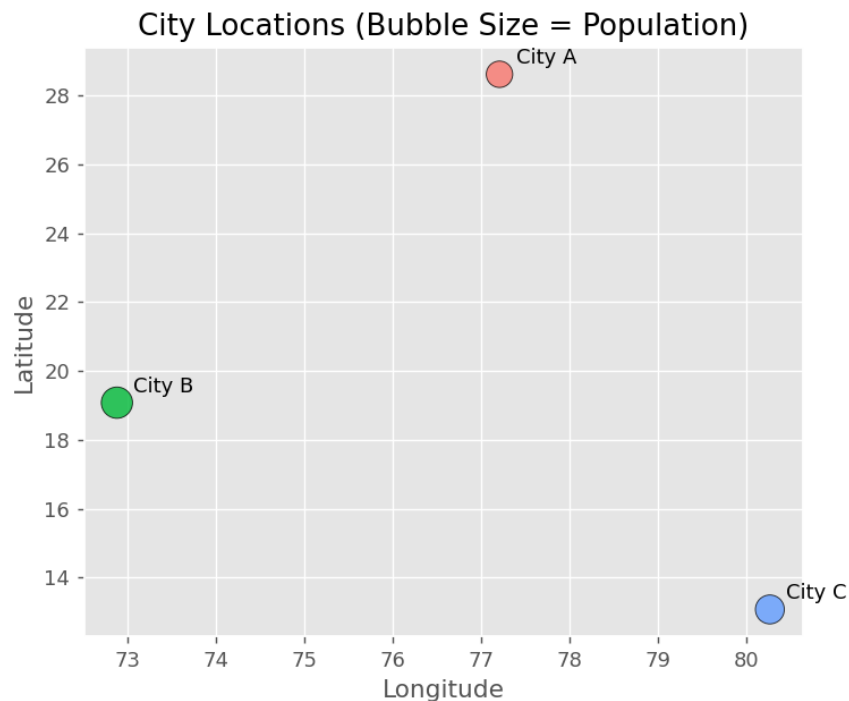
City	Population	Avg. Temperature	Elevation
City A	500000	75	1000
City B	700000	68	800
City C	600000	80	1200

1. Create a map chart to visualize the distribution of cities on a geographic map. Label the map elements.

R-code

```
library(plotly)
geo <- data.frame(
  City = c("City A", "City B", "City C"),
  Population = c(500000, 700000, 600000),
  # ASSUMED illustrative coordinates - not supplied in the dataset
  Lon = c(77.21, 72.88, 80.27),
  Lat = c(28.61, 19.08, 13.08)
)

plot_ly(geo, lat = ~Lat, lon = ~Lon, type = "scattergeo",
  text = ~City, marker = list(size = ~Population/20000, color = "red")) %>%
  layout(title = "City Locations (Bubble Size = Population)",
    geo = list(scope = "asia"))
```



Python code

```
import pandas as pd
import matplotlib.pyplot as plt

geo = pd.DataFrame({
  "City": ["City A", "City B", "City C"],
```

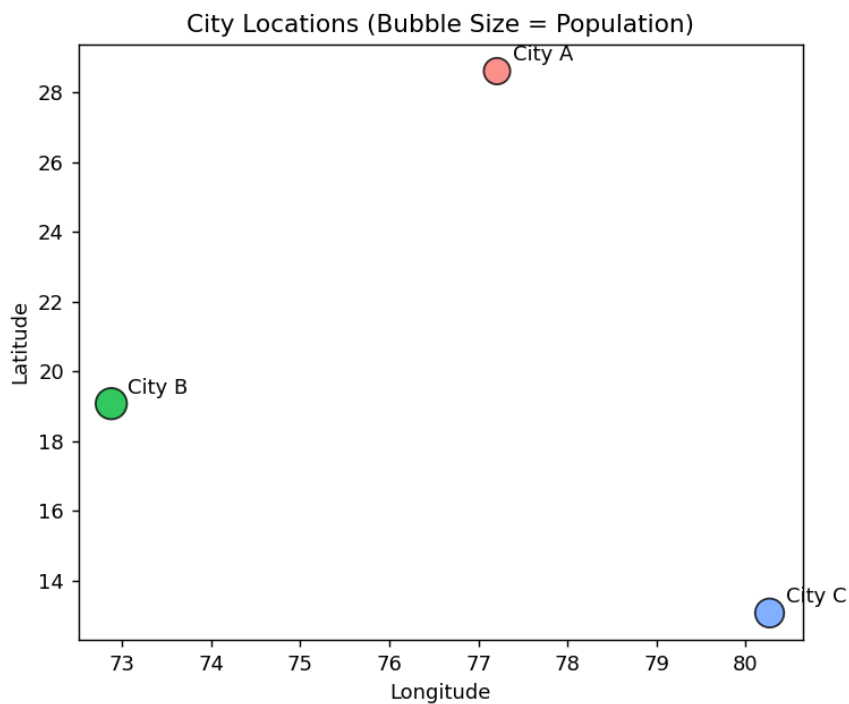


```

    "Population": [500000, 700000, 600000],
    "Lon": [77.21, 72.88, 80.27],    # ASSUMED illustrative coordinates
    "Lat": [28.61, 19.08, 13.08]
  })

plt.figure(figsize=(6,5))
plt.scatter(geo["Lon"], geo["Lat"], s=geo["Population"]/3000, alpha=0.8, edgecolors="black")
for _, row in geo.iterrows():
    plt.annotate(row["City"], (row["Lon"], row["Lat"]), textcoords="offset points", xytext=(8,5))
plt.title("City Locations (Bubble Size = Population)")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.show()

```



2. Generate a scatter plot to explore the relationship between average temperature and population size. Explain any insights.

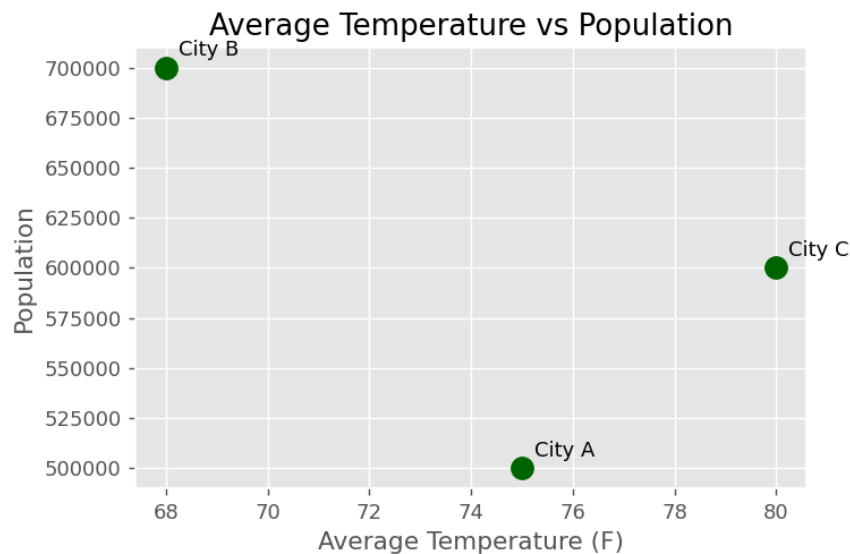
R-code:

```

geo <- data.frame(
  City = c("City A", "City B", "City C"),
  Population = c(500000, 700000, 600000),
  AvgTemperature = c(75, 68, 80)
)

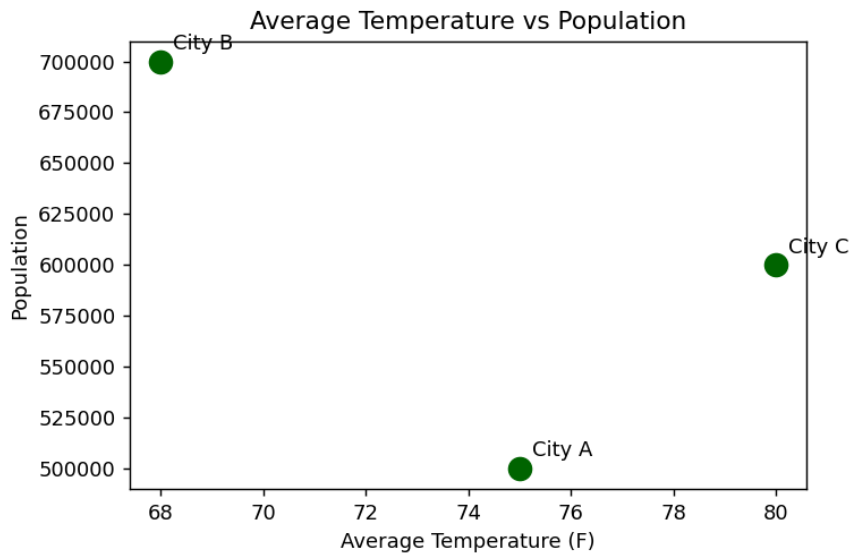
ggplot(geo, aes(x = AvgTemperature, y = Population)) +
  geom_point(size = 4, color = "darkgreen") +
  geom_text(aes(label = City), vjust = -1) +
  labs(title = "Average Temperature vs Population", x = "Avg Temperature (F)", y = "Population") +
  theme_minimal()

```



Python code

```
plt.figure(figsize=(6,4))
plt.scatter(geo["AvgTemperature"], geo["Population"], s=120, color="darkgreen")
for _, row in geo.iterrows():
    plt.annotate(row["City"], (row["AvgTemperature"], row["Population"]), textcoords="offset points",
xytext=(6,6))
plt.title("Average Temperature vs Population")
plt.xlabel("Average Temperature (F)")
plt.ylabel("Population")
plt.show()
```



Insight: City C has the highest average temperature and the lowest elevation, while City B is the coolest and has the largest population, so this small sample does not show a clear/strong relationship between temperature and population size.

3. Build a table to display the geographic data for each city. Label the table elements.

R-code

```
geo_table <- data.frame(
  City = c("City A", "City B", "City C"),
  Population = c(500000, 700000, 600000),
  AvgTemperature = c(75, 68, 80),
  Elevation = c(1000, 800, 1200)
```

```
)  
print(geo_table)
```

Output:

	City	Population	AvgTemperature	Elevation
1	City A	500000	75	1000
2	City B	700000	68	800
3	City C	600000	80	1200

Python code:

```
print(geo[["City", "Population", "AvgTemperature", "Elevation"]])
```

Output:

	City	Population	AvgTemperature	Elevation
0	City A	500000	75	1000
1	City B	700000	68	800
2	City C	600000	80	1200

4. Develop a Tableau dashboard combining the map chart, scatter plot, and the table for interactive exploration of geographic data.

Note: This task is built in Tableau Desktop, not in R or Python. Tableau has native geographic mapping (double-click City to auto-generate a map), so: (1) build a map sheet with Population as size/color, (2) build a scatter-plot sheet of Avg. Temperature vs Population, (3) add a text table sheet, and (4) combine all three on one Dashboard.

Set 14: Survey Results Analysis

Dataset: Survey Results

Respondent	Question 1	Question 2	Question 3
1	A	B	C
2	B	A	D
3	C	A	B

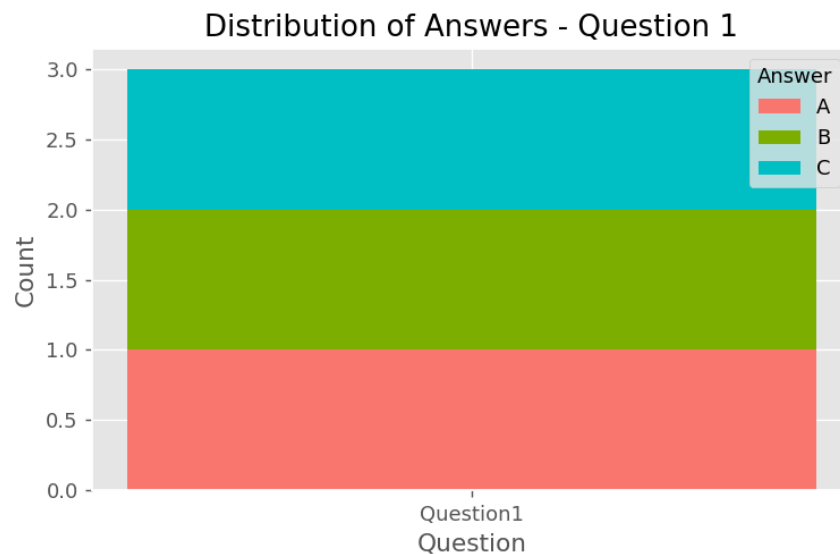
1. Create a stacked bar chart to display the distribution of answers for Question 1. Label the chart elements.

R-code

```
survey <- data.frame(
  Respondent = 1:3,
  Question1 = c("A", "B", "C"),
  Question2 = c("B", "A", "A"),
  Question3 = c("C", "D", "B")
)

q1_data <- as.data.frame(table(survey$Question1))
colnames(q1_data) <- c("Answer", "Count")

ggplot(q1_data, aes(x = "Question1", y = Count, fill = Answer)) +
  geom_bar(stat = "identity", position = "stack") +
  labs(title = "Distribution of Answers - Question 1", x = "Question", y = "Count") +
  theme_minimal()
```



Python code

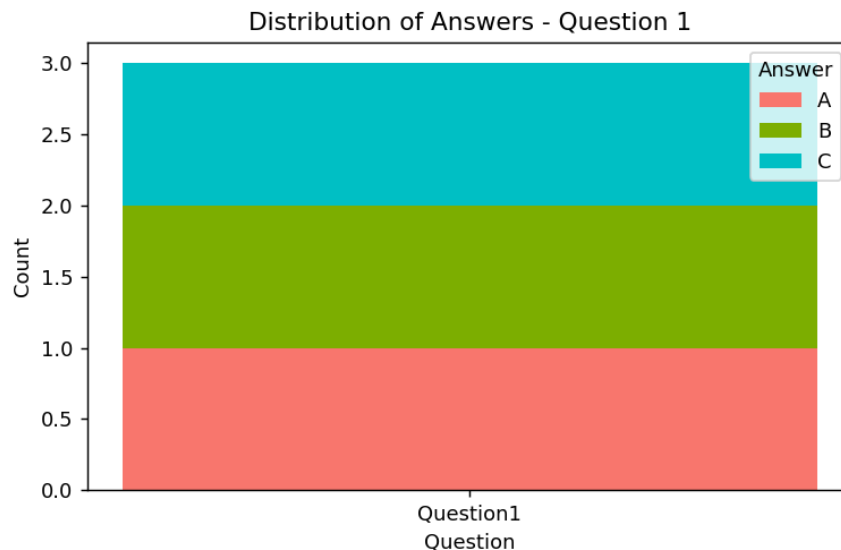
```
import pandas as pd
import matplotlib.pyplot as plt

survey = pd.DataFrame({
  "Respondent": [1, 2, 3],
  "Question1": ["A", "B", "C"],
  "Question2": ["B", "A", "A"],
  "Question3": ["C", "D", "B"]
})
```

```

q1_counts = survey["Question1"].value_counts().reindex(["A", "B", "C", "D"]).fillna(0)
bottom = 0
plt.figure(figsize=(6,4))
for ans, cnt in q1_counts.items():
    if cnt > 0:
        plt.bar("Question1", cnt, bottom=bottom, label=ans)
        bottom += cnt
plt.title("Distribution of Answers - Question 1")
plt.xlabel("Question")
plt.ylabel("Count")
plt.legend(title="Answer")
plt.show()

```



2. Generate a radar chart to visualize the overall pattern of responses across all three questions.

R-code:

```

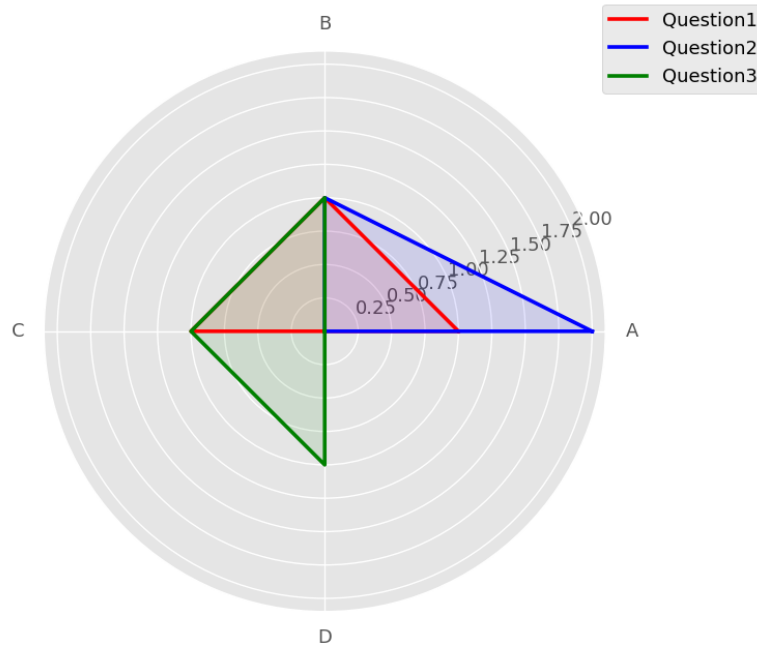
library(fmsb)

answers <- c("A", "B", "C", "D")
radar_data <- sapply(c("Question1", "Question2", "Question3"), function(q) {
  as.numeric(table(factor(survey[[q]], levels = answers)))
})
radar_df <- as.data.frame(t(radar_data))
colnames(radar_df) <- answers
radar_df <- rbind(rep(3, 4), rep(0, 4), radar_df) # max row, min row, then data

radarchart(radar_df, axistype = 1, pcol = c("red", "blue", "green"),
  title = "Response Pattern Across Questions")
legend("topright", legend = c("Question1", "Question2", "Question3"),
  col = c("red", "blue", "green"), lty = 1)

```

Response Pattern Across Questions



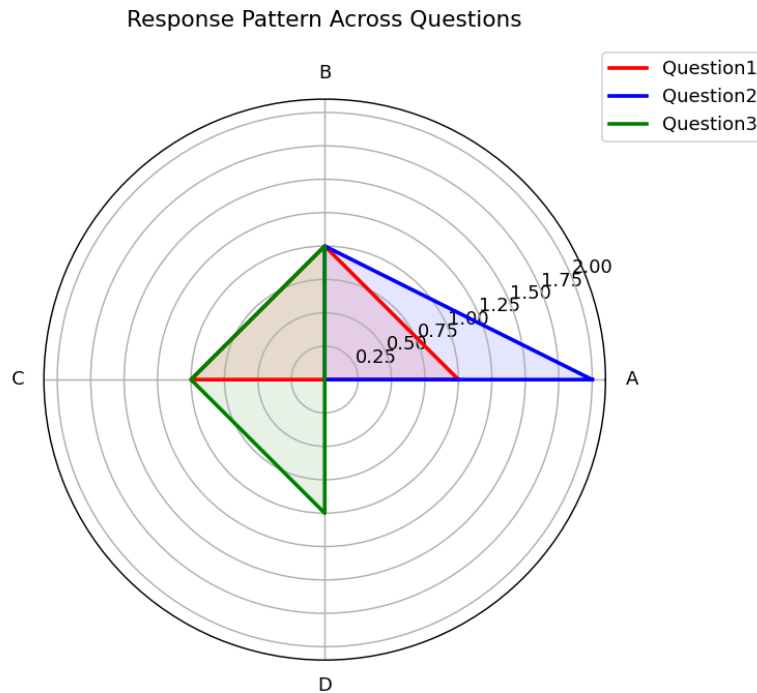
Python code:

```
import numpy as np
import matplotlib.pyplot as plt

answers = ["A", "B", "C", "D"]
questions = ["Question1", "Question2", "Question3"]
data = {q: survey[q].value_counts().reindex(answers).fillna(0).values for q in questions}

angles = np.linspace(0, 2*np.pi, len(answers), endpoint=False).tolist()
angles += angles[:1]

fig, ax = plt.subplots(figsize=(6,6), subplot_kw=dict(polar=True))
colors = ["red", "blue", "green"]
for i, q in enumerate(questions):
    values = data[q].tolist()
    values += values[:1]
    ax.plot(angles, values, color=colors[i], linewidth=2, label=q)
    ax.fill(angles, values, color=colors[i], alpha=0.1)
ax.set_xticks(angles[:-1])
ax.set_xticklabels(answers)
plt.title("Response Pattern Across Questions")
plt.legend(loc="upper right", bbox_to_anchor=(1.3, 1.1))
plt.show()
```



3. Build a table to show the survey response data for each respondent. Label the table elements.

R-code:

```
print(survey)
```

Output:

Respondent	Question1	Question2	Question3	
1	1	A	B	C
2	2	B	A	D
3	3	C	A	B

Python code:

```
print(survey)
```

Output:

Respondent	Question1	Question2	Question3	
0	1	A	B	C
1	2	B	A	D
2	3	C	A	B

4. Develop a Tableau dashboard combining the stacked bar chart, radar chart, and the table for interactive exploration of survey responses.

Note: This task is built in Tableau Desktop, not in R or Python. Steps: (1) build a stacked-bar sheet of Question1 answer counts, (2) build a radar/spider-style sheet (Tableau needs a polygon calculation for true radar charts), (3) add a text table sheet, and (4) combine all three on one Dashboard.

Set 15: Product Sales Analysis

Dataset: Monthly Product Sales

Product ID	Product Name	January Sales	February Sales	March Sales
1	Product A	2000	2200	2400
2	Product B	1500	1800	1600
3	Product C	1200	1400	1100

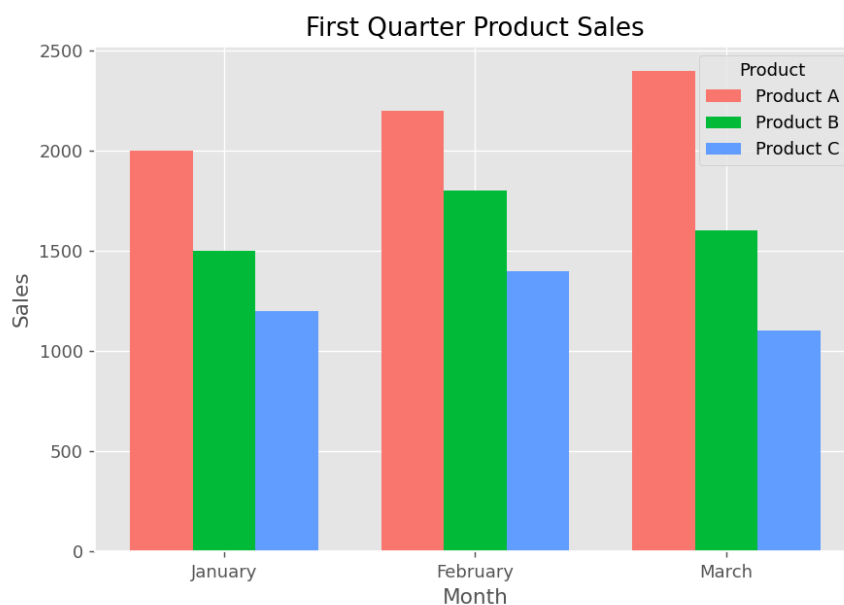
1. Create a grouped bar chart to visualize the sales of each product for the first quarter. Label the chart elements.

R-code:

```
sales <- data.frame(
  ProductID = c(1,2,3),
  Product_Name = c("Product A","Product B","Product C"),
  January = c(2000,1500,1200),
  February = c(2200,1800,1400),
  March = c(2400,1600,1100)
)

library(reshape2)
sales_long <- melt(sales, id.vars = c("ProductID","Product_Name"),
  variable.name = "Month", value.name = "Sales")

ggplot(sales_long, aes(x = Month, y = Sales, fill = Product_Name)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(
    title = "First Quarter Product Sales",
    x = "Month",
    y = "Sales",
    fill = "Product"
  ) +
  theme_minimal()
```



Python code:

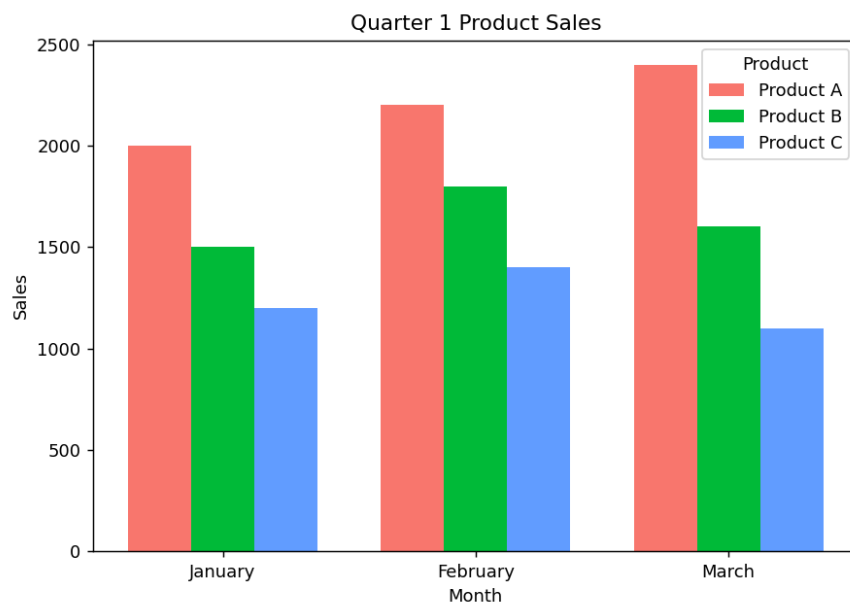

```

import pandas as pd
import matplotlib.pyplot as plt

sales = pd.DataFrame({
    "ProductID": [1, 2, 3],
    "Product": ["Product A", "Product B", "Product C"],
    "January": [2000, 1500, 1200],
    "February": [2200, 1800, 1400],
    "March": [2400, 1600, 1100]
})

sales.set_index("Product")["January", "February", "March"].plot(
    kind="bar",
    figsize=(8,5)
)
plt.title("Quarter 1 Product Sales")
plt.xlabel("Product Name")
plt.ylabel("Sales")
plt.legend(title="Month")
plt.show()

```



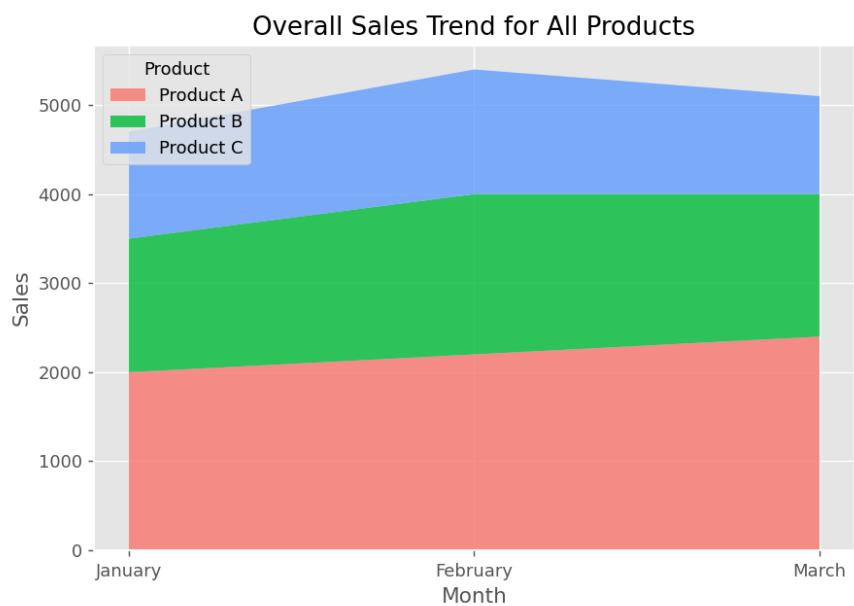
2. Generate a stacked area chart to represent the overall sales trend for all products over the three months.

R-code:

```

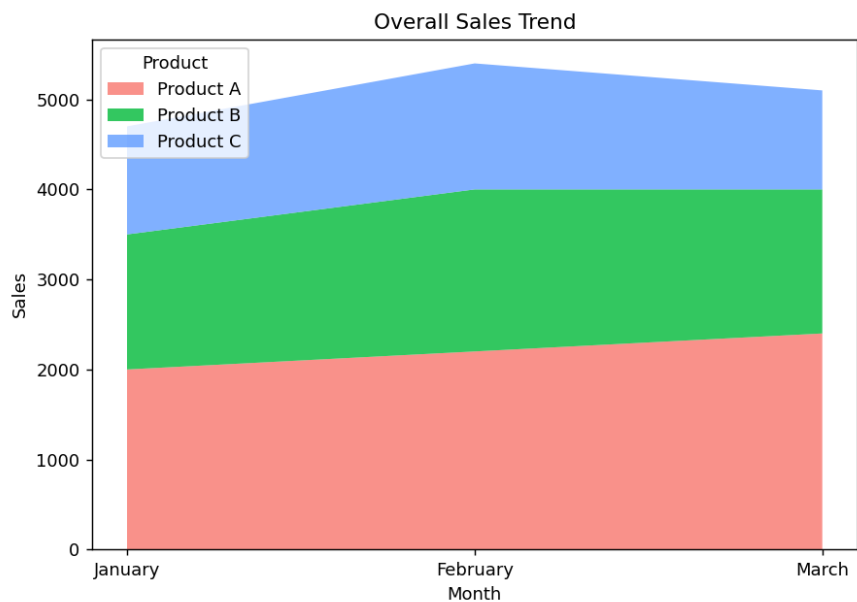
ggplot(sales_long, aes(x = Month, y = Sales, fill = Product_Name, group = Product_Name)) +
  geom_area(alpha = 0.7) +
  labs(
    title = "Overall Sales Trend for All Products",
    x = "Month",
    y = "Sales",
    fill = "Product"
  ) +
  theme_minimal()

```



Python code:

```
area = sales.set_index("Product").T
area.plot(
    kind="area",
    stacked=True,
    figsize=(8,5)
)
plt.title("Overall Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```



3. Build a table to show the monthly sales data for each product. Label the table elements.

R-code:

```
sales <- data.frame(
  ProductID = c(1,2,3),
  Product = c("Product A", "Product B", "Product C"),
  January = c(2000,1500,1200),
  February = c(2200,1800,1400),
```

```
    March = c(2400,1600,1100)
  )
print(sales)
```

Output:

	ProductID	Product	January	February	March
1	1	Product A	2000	2200	2400
2	2	Product B	1500	1800	1600
3	3	Product C	1200	1400	1100

Python code:

```
print(sales)
```

Output:

	ProductID	Product	January	February	March
0	1	Product A	2000	2200	2400
1	2	Product B	1500	1800	1600
2	3	Product C	1200	1400	1100

4. Develop a Tableau dashboard combining the grouped bar chart, stacked area chart, and the table for interactive exploration of sales data.

Note: This task is built in Tableau Desktop, not in R or Python. Steps: (1) build a grouped-bar sheet of Q1 sales by product/month, (2) build a stacked-area sheet of overall sales trend, (3) add a text table sheet, and (4) combine all three on one Dashboard with a Product filter.